Search Module — Design & Architecture

Version: 1.0
Date: April 15, 2026
Author: Architecture Team
Parent Doc: banking-application-architecture.md

Table of Contents

1. Problem Statement
2. Search Requirements Matrix
3. High-Level Architecture
4. Search Data Pipeline
5. Elasticsearch Index Design
6. Search API Design
7. Query Strategy — When to Hit What
8. Relevance Scoring & Ranking
9. Typeahead / Autocomplete
10. Search Security & Data Isolation
11. Performance Optimization
12. Failure Handling & Fallback
13. GraphQL vs REST — Why REST for Search
14. Monitoring & Observability

---

1. Problem Statement

A banking application generates millions of transactions daily across multiple accounts, payment types, and categories. Users need to search through this data with sub-second response times, using partial keywords, descriptions, merchant names, reference IDs, and amounts. Simple `LIKE '%keyword%'` queries on PostgreSQL at this scale cause full table scans, degrade DB performance, and block the primary transactional workload.

Core challenge: The `transactions` table is range-partitioned by month and grows to billions of rows. We need full-text search across descriptions, fuzzy matching for misspelled merchant names, and near real-time indexing — without impacting the primary OLTP database.

2. Search Requirements Matrix

Requirement	Detail	Priority
Full-text search	Search across transaction descriptions, merchant names, reference IDs	P0
Fuzzy matching	Tolerate typos: "Amzon" → "Amazon", "Swggy" → "Swiggy"	P0
Sub-second latency	p99 < 200ms for search queries	P0
Real-time indexing	New transactions searchable within 5 seconds of creation	P0
Scoped search	Users can only search their own transactions (strict tenant isolation)	P0
Combined with filters	Search works alongside date range, category, amount, status filters	P0
Autocomplete	Typeahead suggestions as user types (debounced 300ms)	P1
Highlighted results	Matching terms highlighted in search results	P1
Search analytics	Track popular search terms, zero-result queries	P2
Synonym support	"UPI" matches "Unified Payments Interface", "transfer" matches "payment"	P2

3. High-Level Architecture

```
                          SEARCH ARCHITECTURE
                          ___________________

┌──────────────┐          ┌──────────────┐          ┌──────────────────────┐
│   React UI   │──────────►│   BFF-Web    │──────────►│  Transaction Service │
│              │          │              │          │                      │
│ SearchBar    │          │ GET /search  │          │ /api/v1/transactions │
│ Autocomplete │          │              │          │   /search            │
│ FilterPanel  │          └──────────────┘          └──────────────────────┘
└──────────────┘                                              │
                                                              │
                                                   ┌──────────────────────┐
                                                   │     Search Router    │
                                                   │                      │
                                                   │  Decides: ES or PG?  │
                                                   │  (based on query     │
                                                   │   complexity)        │
                                                   └──────────────────────┘
                                                          │       │
                                                   ┌──────┘       └──────┐
                                                   ▼                     ▼
                                       ┌──────────────────┐   ┌──────────────────┐
                                       │  Elasticsearch   │   │   PostgreSQL     │
                                       │                  │   │                  │
                                       │  • Full-text     │   │  • Simple lookups │
                                       │  • Fuzzy match   │   │  • Exact ID match │
                                       │  • Autocomplete  │   │  • Filter-only   │
                                       │  • Aggregations  │   │    (no text query)│
                                       └──────────────────┘   └──────────────────┘

                                 ┌──────────────────────────────┐
                                 │    INDEXING PIPELINE (Async)  │
                                 │                              │
                                 │  Kafka (txn-events)          │
                                 │        │                     │
                                 │        ▼                     │
                                 │  ES Index Consumer           │
                                 │  (reads events, writes to ES) │
                                 │                              │
                                 │  Lag target: < 5 seconds     │
                                 └──────────────────────────────┘
```

4. Search Data Pipeline

New transactions must be searchable within seconds. We use Kafka as the bridge between the source-of-truth (PostgreSQL) and the search index (Elasticsearch).

4.1 Indexing Flow

```
Transaction        Kafka             ES Index          Elasticsearch
Service            (txn-events)      Consumer          Cluster
   |                  |                |                  |
   |── INSERT into PG |                |                  |
   |── PUBLISH event ─►|               |                  |
   |                  |── Consume ────►|                  |
   |                  |                |── Transform      |
   |                  |                |   (extract fields, |
   |                  |                |    mask PII,     |
   |                  |                |    enrich data)  |
   |                  |                |── Bulk Index ────►|
   |                  |                |                  |── Indexed
   |                  |                |                  |
   |                  |                |   Batch: 500 docs |
   |                  |                |   or every 2s    |
   |                  |                |   (whichever first)|
```

4.2 Why Kafka-Based Indexing (Not Dual Writes)

Approach	Problem
Dual write (PG + ES in same request)	If ES write fails, data is inconsistent. If we make it transactional, latency doubles and availability halves.
CDC (Debezium)	Viable but adds operational complexity (WAL parsing, connector management). Overkill when we already have Kafka events.
Kafka consumer (chosen)	Already publishing `txn-events` for notifications, fraud detection, audit. Adding one more consumer group is trivial. Decoupled, retry-friendly, exactly-once semantics via idempotent writes.

4.3 Document Transformation

The ES Index Consumer transforms raw Kafka events into search-optimized documents:

```java
@KafkaListener(topics = "txn-events", groupId = "es-indexer-cg")
public void onTransactionEvent(TransactionEvent event) {
    if (event.getType() == TRANSACTION_COMPLETED || event.getType() == TRANSACTION_FAIL

        SearchDocument doc = SearchDocument.builder()
            .id(event.getTransactionId())
            .accountId(event.getAccountId())
            .userId(event.getUserId())                          // for tenant isolation
            .amount(event.getAmount())
            .currency(event.getCurrency())
            .type(event.getTransactionType())                   // CREDIT / DEBIT
            .category(event.getCategory())                      // UPI / NEFT / RTGS / E
            .status(event.getStatus())                          // SUCCESS / FAILED
            .description(event.getDescription())                // "Paid to Swiggy - Ord
            .merchantName(extractMerchant(event))               // "Swiggy"
            .referenceId(event.getReferenceId())
            .tags(deriveTags(event))                            // ["food", "delivery",
            .createdAt(event.getTimestamp())
            .searchableText(buildSearchableText(event))         // combined text for _al
            .build();

        bulkIndexer.add(doc);
    }
}
```

4.4 Bulk Indexing Strategy

Individual index calls to ES are expensive. We batch them:

- **Batch size:** 500 documents
- **Flush interval:** 2 seconds (whichever threshold is hit first)
- **Retry on failure:** 3 attempts with exponential backoff (1s, 5s, 30s)
- **Dead letter:** Failed documents written to `es-index.DLQ` Kafka topic for manual reprocessing

4.5 Full Re-Index Strategy

When the ES mapping changes or index corruption occurs:

1. Create a new index (`transactions-v2`) with the updated mapping.
2. Run a re-index job that reads from PostgreSQL read replica (paginated, 10K rows/batch).
3. Use the Elasticsearch `_reindex` API or a custom batch job.
4. Once caught up, swap the index alias `transactions-current` from `transactions-v1` → `transactions-v2`.
5. Delete the old index.
6. Zero downtime — reads continue hitting the alias throughout.

```
transactions-current (alias)
      │
      ├── transactions-v1  (old, serving reads during re-index)
      │
      └── transactions-v2  (new, being populated)

After swap:
transactions-current (alias) ──▶ transactions-v2
transactions-v1 ──▶ deleted
```

---

5. Elasticsearch Index Design

5.1 Index Mapping

```
{
  "settings": {
    "number_of_shards": 6,
    "number_of_replicas": 1,
    "analysis": {
      "analyzer": {
        "transaction_analyzer": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "synonym_filter", "edge_ngram_filter"
        },
        "autocomplete_analyzer": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "edge_ngram_filter"]
        },
        "search_analyzer": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "synonym_filter"]
        }
      },
      "filter": {
        "edge_ngram_filter": {
          "type": "edge_ngram",
          "min_gram": 2,
          "max_gram": 15
        },
        "synonym_filter": {
          "type": "synonym",
          "synonyms": [
            "upi, unified payments interface",
            "neft, national electronic fund transfer",
            "rtgs, real time gross settlement",
            "emi, equated monthly installment",
            "atm, automated teller machine",
            "txn, transaction, transfer, payment"
          ]
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "id":              { "type": "keyword" },
      "accountId":       { "type": "keyword" },
      "userId":          { "type": "keyword" },
      "amount":          { "type": "scaled_float", "scaling_factor": 100 },
```

```
      "currency":        { "type": "keyword" },
      "type":            { "type": "keyword" },
      "category":        { "type": "keyword" },
      "status":          { "type": "keyword" },
      "description": {
        "type": "text",
        "analyzer": "transaction_analyzer",
        "search_analyzer": "search_analyzer",
        "fields": {
          "autocomplete": {
            "type": "text",
            "analyzer": "autocomplete_analyzer",
            "search_analyzer": "standard"
          },
          "exact": {
            "type": "keyword"
          }
        }
      },
      "merchantName": {
        "type": "text",
        "analyzer": "transaction_analyzer",
        "search_analyzer": "search_analyzer",
        "fields": {
          "keyword": { "type": "keyword" },
          "autocomplete": {
            "type": "text",
            "analyzer": "autocomplete_analyzer",
            "search_analyzer": "standard"
          }
        }
      },
      "referenceId":     { "type": "keyword" },
      "tags":            { "type": "keyword" },
      "createdAt":       { "type": "date" },
      "searchableText": {
        "type": "text",
        "analyzer": "transaction_analyzer",
        "search_analyzer": "search_analyzer"
      }
    }
  }
}
```

5.2 Shard Sizing Rationale

Factor	Value	Reasoning
Shards	6 primary	~50M docs/shard target. At 300M txns/year, a single index covers 1 year comfortably. Allows parallel query execution across 6 shards.
Replicas	1	Doubles read throughput. Acceptable storage overhead for search nodes.
Index rollover	Monthly alias	`transactions-2026-04`, `transactions-2026-05`, etc. Queries span multiple indices via alias `transactions-*`. Old indices can be force-merged and moved to warm storage.

5.3 Index Lifecycle Management (ILM)

```
Hot (0-3 months)     → Primary SSD nodes, full replicas, actively indexed
Warm (3-12 months)   → Moved to warm nodes, force-merged to 1 segment/shard, read-only
Cold (12-36 months)  → Frozen index on cold storage, searchable but slow
Delete (36+ months)  → Deleted (PG remains the source of truth for compliance)
```

---

6. Search API Design

6.1 Endpoint

```
GET /api/v1/transactions/search
    ?q=swiggy
    &accountIds=ACC-001,ACC-002
    &from=2026-01-01
    &to=2026-04-15
    &categories=UPI,NEFT
    &minAmount=100
    &maxAmount=5000
    &status=SUCCESS
    &page=0
    &size=20
    &sort=relevance
```

6.2 Response

```
{
  "results": [
    {
      "id": "txn-456",
      "accountId": "ACC-001",
      "amount": 450.00,
      "type": "DEBIT",
      "category": "UPI",
      "status": "SUCCESS",
      "description": "Paid to <em>Swiggy</em> - Order #1234",
      "merchantName": "Swiggy",
      "createdAt": "2026-04-14T18:30:00Z",
      "highlights": {
        "description": ["Paid to <em>Swiggy</em> - Order #1234"],
        "merchantName": ["<em>Swiggy</em>"]
      }
    }
  ],
  "pagination": {
    "page": 0,
    "size": 20,
    "totalElements": 47,
    "totalPages": 3
  },
  "aggregations": {
    "byCategory": { "UPI": 32, "NEFT": 10, "RTGS": 5 },
    "byStatus": { "SUCCESS": 45, "FAILED": 2 },
    "totalAmount": 127500.00
  },
  "meta": {
    "took": 45,
    "searchEngine": "elasticsearch",
    "queryId": "qry-abc-123"
  }
}
```

6.3 Search vs. Filter-Only Routing

The Search Router decides where to send the query:

```
┌──────────────────────────────────────────────────────────────┐
│                    SEARCH ROUTER LOGIC                       │
│                                                              │
│  if (q is present AND q.length ≥ 2) {                        │
│      // Text search → must go to Elasticsearch               │
│      route to ES with filters as post_filter                 │
│  }                                                           │
│  else if (only structured filters, no text query) {          │
│      // Pure filter → PostgreSQL is faster (indexed cols)    │
│      route to PG with JPA Specification                      │
│  }                                                           │
│  else if (exact referenceId or transactionId lookup) {       │
│      // Direct PK/unique key lookup → PostgreSQL             │
│      route to PG (index scan, microseconds)                  │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
```

Why not always use Elasticsearch?

- Pure filter queries on indexed PostgreSQL columns (accountId + date range + category) are faster and cheaper than ES for structured queries.
- ES shines for full-text, fuzzy, and relevance-scored queries.
- Routing to PG for simple filters reduces load on the ES cluster.

---

7. Query Strategy — When to Hit What

Query Type	Engine	Example	Why
Free-text search	Elasticsearch	`q=grocery store`	Full-text analysis, fuzzy matching, relevance scoring
Fuzzy / typo-tolerant	Elasticsearch	`q=Amzon` → Amazon	Levenshtein distance matching built into ES
Autocomplete suggestions	Elasticsearch	`q=sw` → Swiggy, Swipe	Edge n-gram analyzed field, prefix queries
Filter-only (structured)	PostgreSQL	`category=UPI&status=SUCCESS&from=2026-01-01`	B-tree indexes on (account_id, created_at, category) are optimal
Exact ID lookup	PostgreSQL	`referenceId=REF-12345`	Primary key / unique index scan
Aggregation (dashboard)	Elasticsearch	"Total UPI spend this month"	ES aggregation framework is purpose-built for this
Export / bulk download	PostgreSQL (read replica)	"Download all transactions as CSV"	Sequential scan, no relevance needed

8. Relevance Scoring & Ranking

8.1 Elasticsearch Query (bool + function_score)

```
{
  "query": {
    "function_score": {
      "query": {
        "bool": {
          "must": [
            {
              "multi_match": {
                "query": "swiggy food",
                "fields": [
                  "description^3",
                  "merchantName^5",
                  "referenceId^2",
                  "searchableText",
                  "tags"
                ],
                "type": "best_fields",
                "fuzziness": "AUTO",
                "prefix_length": 2
              }
            }
          ],
          "filter": [
            { "terms": { "accountId": ["ACC-001", "ACC-002"] } },
            { "range": { "createdAt": { "gte": "2026-01-01", "lte": "2026-04-15" } } },
            { "terms": { "category": ["UPI"] } },
            { "range": { "amount": { "gte": 100, "lte": 5000 } } },
            { "term": { "status": "SUCCESS" } }
          ]
        }
      },
      "functions": [
        {
          "gauss": {
            "createdAt": {
              "origin": "now",
              "scale": "30d",
              "decay": 0.5
            }
          },
          "weight": 1.5
        }
      ],
      "boost_mode": "multiply"
    }
  },
  "highlight": {
    "fields": {
```

```
      "description": { "pre_tags": ["<em>"], "post_tags": ["</em>"] },
      "merchantName": { "pre_tags": ["<em>"], "post_tags": ["</em>"] }
    }
  },
  "aggs": {
    "byCategory": { "terms": { "field": "category" } },
    "byStatus": { "terms": { "field": "status" } },
    "totalAmount": { "sum": { "field": "amount" } }
  }
}
```

8.2 Field Boosting Rationale

Field	Boost	Reasoning
`merchantName`	5x	Users most often search by merchant: "Swiggy", "Amazon", "Zomato"
`description`	3x	Contains merchant + order details: "Paid to Swiggy - Order #1234"
`referenceId`	2x	Exact reference lookups are high-intent (support/dispute context)
`searchableText`	1x (default)	Catch-all fallback field
`tags`	1x	Derived tags ("food", "travel") match broad searches

8.3 Recency Bias

The `gauss` function on `createdAt` decays relevance scores for older transactions. A search for "Swiggy" ranks yesterday's Swiggy order higher than one from 6 months ago. The decay is gentle (scale=30d, decay=0.5), so recent transactions score ~1.5x compared to 3-month-old ones.

---

9. Typeahead / Autocomplete

9.1 Architecture

```
User types "sw"
     |
     ▼ (debounced 300ms)
GET /api/v1/transactions/autocomplete?q=sw&accountIds=ACC-001
     |
     ▼
Elasticsearch (autocomplete field, edge n-gram)
     |
     ▼
Response:
{
  "suggestions": [
    { "text": "Swiggy",           "type": "merchant", "count": 23 },
    { "text": "Swipe Card Fee",   "type": "description", "count": 3 },
    { "text": "SWI-REF-9876",     "type": "referenceId", "count": 1 }
  ]
}
```

9.2 Implementation

Uses an edge n-gram analyzer (min_gram=2) so "sw" matches "Swiggy", "Swipe", "SWI-REF". The autocomplete endpoint runs a lightweight `prefix` + `bool` filter query limited to 5 suggestions to keep latency under 50ms.

9.3 Frontend Behavior

```
function SearchBar() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 300);

  const { data: suggestions } = useQuery(
    ['autocomplete', debouncedQuery],
    () => searchApi.autocomplete(debouncedQuery),
    {
      enabled: debouncedQuery.length ≥ 2,
      staleTime: 60_000,
      keepPreviousData: true
    }
  );

  return (
    <Combobox>
      <Combobox.Input value={query} onChange={setQuery} placeholder="Search transaction
      <Combobox.Options>
        {suggestions?.map(s => (
          <Combobox.Option key={s.text} value={s.text}>
            <HighlightedText text={s.text} highlight={query} />
            <Badge>{s.type}</Badge>
            <span className="count">{s.count} transactions</span>
          </Combobox.Option>
        ))}
      </Combobox.Options>
    </Combobox>
  );
}
```

---

10. Search Security & Data Isolation

10.1 Tenant Isolation

Every search query is scoped by `userId` (extracted from the JWT). This is enforced at the service layer, not the client.

```java
public Page<SearchResult> search(TransactionSearchDTO request, String userId) {
    BoolQueryBuilder query = QueryBuilders.boolQuery()
        .filter(QueryBuilders.termQuery("userId", userId));  // mandatory tenant filter

    if (hasText(request.getQuery())) {
        query.must(buildTextQuery(request.getQuery()));
    }

    // ... other filters added to query.filter()
    return esClient.search(query);
}
```

The `userId` filter is applied as a Kafka `filter` clause (not `must`), so it does not affect relevance scoring but strictly limits results to the authenticated user's data.

10.2 PII in Search Index

Field	Indexed?	Handling
Transaction description	Yes	Descriptions are user-facing and non-sensitive
Merchant name	Yes	Public merchant names, not PII
Account number	No	Not indexed in ES. Only `accountId` (UUID) is stored
User name / email	No	Never sent to ES
Aadhaar / PAN	No	Never indexed anywhere outside encrypted PG columns

10.3 Admin Search

Admin users (with `ADMIN` or `AUDITOR` role) can search across all users' transactions. The service layer checks the role and conditionally removes the `userId` filter, adding an audit trail event for every admin search query.

---

11. Performance Optimization

11.1 Caching Search Results

Layer	Cache	TTL	Rationale
RTK Query (frontend)	In-memory	30s (`staleTime`)	Avoid redundant API calls while user paginates/tweaks filters
BFF response cache	None	—	Search results are personalized; caching at BFF adds complexity without benefit
Redis (backend)	Autocomplete popular terms	5 min	Top 100 autocomplete queries cached to reduce ES load
Elasticsearch	Request cache (built-in)	Auto-invalidated on index refresh	ES caches shard-level results for repeated identical queries

11.2 Query Performance Techniques

Technique	Detail
Filters in filter context	Structured filters (accountId, date, category) go in `bool.filter`, not `bool.must`. Filter context skips scoring and is cacheable by ES.
Routing by userId	All documents for a user land on the same shard (`routing=userId`). Search hits a single shard instead of broadcasting to all 6.
Index aliases for time ranges	`from=2026-01-01&to=2026-03-31` resolves to querying only `transactions-2026-01`, `transactions-2026-02`, `transactions-2026-03` instead of all indices.
Pre-filter with date range	Date range filter prunes 90%+ of documents before text matching runs. Always applied first.
Limit returned fields	ES `_source` filtering returns only displayed fields, reducing network transfer.

11.3 Latency Budget

```
Total search latency (p99): < 200ms

┌──────────────────────────────────────┐
│ BFF overhead          │     10ms     │
│ Network (BFF → Svc)   │      5ms     │
│ Auth/validation       │      5ms     │
│ ES query execution    │    120ms     │ ← bulk of the time
│ Response mapping      │     10ms     │
│ Network (Svc → BFF)   │      5ms     │
│ BFF response assembly │      5ms     │
│ ──────────────────────┼──────────────│
│ Total                 │    160ms (p95) │
│                       │    200ms (p99) │
└──────────────────────────────────────┘
```

12. Failure Handling & Fallback

12.1 What Happens When Elasticsearch Is Down?

```
┌──────────────────────────────────────────────────────────┐
│                    ES FAILURE FALLBACK                    │
│                                                           │
│  Search request arrives                                   │
│        │                                                  │
│        ▼                                                  │
│  Circuit breaker check (ES circuit)                       │
│        │                                                  │
│   ┌────┴────┐                                             │
│   │ CLOSED? │── YES ──▶ Route to ES as normal             │
│   └────┬────┘                                             │
│        │ NO (OPEN)                                        │
│        ▼                                                  │
│  Fallback to PostgreSQL:                                  │
│  • Text search → PG full-text search (to_tsvector)        │
│  • Fuzzy match → DISABLED (PG can't do it efficiently)    │
│  • Autocomplete → DISABLED                                │
│  • Structured filters → work normally via JPA Specifications│
│                                                           │
│  Response includes header:                                │
│  X-Search-Degraded: true                                  │
│  X-Search-Engine: postgresql-fallback                     │
│                                                           │
│  Frontend shows banner:                                   │
│  "Search is temporarily limited. Fuzzy and autocomplete   │
│   features are unavailable."                              │
└──────────────────────────────────────────────────────────┘
```

12.2 Indexing Lag Handling

If the ES indexing consumer falls behind Kafka:

- **Lag < 30s:** No action. Users won't notice.
- **Lag 30s–5min:** Alert to Slack. Consumer auto-scales (increase partition consumers).
- **Lag > 5min:** PagerDuty alert. Investigate. Meanwhile, very recent transactions may not appear in search but are always visible in the filter-only (PG-backed) transaction list.
- **Lag > 30min:** Trigger full re-index from PG read replica as a recovery measure.

13. GraphQL vs REST — Why REST for Search

13.1 Why GraphQL Was Evaluated

GraphQL is a natural candidate for the BFF layer. The banking dashboard aggregates data from multiple services (balance, recent transactions, loan EMI) into a single screen. GraphQL's ability to let the client specify exactly which fields it needs — and fetch from multiple "resolvers" in a single query — aligns well with this aggregation pattern.

```
# What a GraphQL search query COULD look like
query TransactionSearch($filters: TransactionFilterInput!, $page: Int, $size: Int) {
  transactionSearch(filters: $filters, page: $page, size: $size) {
    results {
      id
      amount
      description
      category
      status
      createdAt
      highlights {
        description
        merchantName
      }
    }
    pagination {
      totalElements
      totalPages
    }
    aggregations {
      byCategory { key count }
      byStatus { key count }
    }
  }
}
```

13.2 Why We Chose REST Over GraphQL

Concern	REST (Chosen)	GraphQL
Caching	HTTP caching works natively (CDN, browser, API Gateway). `Cache-Control` headers, ETags, and conditional requests are battle-tested.	GraphQL uses POST for all queries — HTTP caching doesn't work out of the box. Requires specialized tools (Apollo Cache, persisted queries) and custom cache key derivation.
Security surface area	Fixed endpoints with known query shapes. Easy to rate-limit per endpoint. WAF rules straightforward.	Arbitrary query depth/complexity. A malicious client can craft deeply nested or computationally expensive queries. Requires query cost analysis, depth limiting, and complexity budgeting. In banking, this expanded attack surface is a risk.
Rate limiting	Per-endpoint: `GET /search` = 100 req/min per user. Simple, predictable.	Per-query complexity: "this query costs 5 points, that one costs 50." Much harder to implement and reason about.
Observability	Each endpoint has clear metrics: `/search` p99 = 180ms. Easy to alert on.	All traffic hits `/graphql`. Distinguishing "search queries" from "balance queries" requires custom instrumentation on operation names.
Error handling	HTTP status codes (400, 404, 429, 500) are universal. Clients, load balancers, and monitoring tools understand them.	GraphQL always returns 200 with errors in the response body. Monitoring tools, CDNs, and WAFs don't see the error. Requires custom error extraction.
File upload / download	Native support (multipart, streaming). Statement PDF download is straightforward.	Requires workarounds (multipart-request spec or separate REST endpoint alongside GraphQL).
Team expertise	Spring Boot REST ecosystem is mature. OpenAPI (Swagger) auto-generates docs, client SDKs, and contract tests (Pact).	Spring Boot GraphQL support exists (`spring-graphql`) but is younger. Fewer tools for contract testing, API documentation, and client code generation compared to OpenAPI.
N+1 on the backend	N/A — BFF makes explicit parallel gRPC calls. Developer controls what gets fetched.	GraphQL resolvers risk N+1 queries unless DataLoader batching is implemented carefully. Each nested field can trigger a resolver, which triggers a downstream call.
Search-specific concerns	Search response structure (results + highlights + aggregations + pagination) is fixed. No benefit from field-level selection — clients always need all parts.	GraphQL shines when different clients need different subsets of data. For search, the response shape is stable — the flexibility is unnecessary.

13.3 Where GraphQL WOULD Make Sense

GraphQL would be reconsidered if:

Scenario	Why GraphQL Helps
Mobile vs Web need radically different fields	Mobile wants 5 fields, web wants 20. GraphQL avoids over-fetching. Currently, BFF-Mobile already handles this via separate BFF services with tailored payloads.
Rapid frontend iteration	If the frontend team wants to add/remove fields without backend deployment, GraphQL's schema-first approach helps. Currently, BFF deployments are fast enough (CI/CD in minutes).
Public API for third parties	If we expose a developer API, GraphQL's self-documenting schema (introspection) and flexible queries reduce the need for versioning. Currently, third-party access is via REST with OpenAPI docs.

13.4 Decision

REST for client-facing APIs, gRPC for internal service-to-service calls. GraphQL is not used. The BFF pattern with two separate BFF services (web and mobile) achieves the same benefits (tailored payloads, aggregation) without GraphQL's caching, security, and observability trade-offs.

If a future requirement introduces a public developer API or the number of client-specific BFF variants grows beyond 3, GraphQL should be re-evaluated as the BFF's external-facing protocol (while keeping gRPC internal).

---

14. Monitoring & Observability

13.1 Key Metrics

Metric	Source	Alert Threshold
Search latency (p99)	Micrometer → Prometheus	> 500ms for 2 min
ES cluster health	ES `_cluster/health`	Status != GREEN for 5 min
ES indexing lag	Kafka consumer lag metric	> 10,000 messages for 5 min
Zero-result search rate	Application metric	> 30% of queries return 0 results (indicates bad synonyms or missing data)
ES query error rate	ES slow log + app metrics	> 1% for 2 min
Shard size	ES `_cat/shards`	> 50GB per shard
ES JVM heap usage	ES `_nodes/stats`	> 75% for 10 min

13.2 Search Analytics (P2)

Every search query is logged (with PII stripped) to a `search-analytics` Kafka topic:

```
{
  "queryId": "qry-abc-123",
  "query": "swiggy",
  "filters": { "category": ["UPI"], "dateRange": "30d" },
  "resultCount": 23,
  "latencyMs": 85,
  "engine": "elasticsearch",
  "userId": "user-007-hash",
  "timestamp": "2026-04-15T10:30:00Z"
}
```

Consumed by the analytics pipeline to power:

- **Popular search terms dashboard** (helps optimize synonyms)
- **Zero-result query report** (identifies gaps — should we add "grocery" as a tag?)
- **Search latency percentile charts**

End of Document — Search Module v1.1